



Módulo: Desarrollo de Aplicaciones Web

Introducción a la Programación 2 — Ingeniería de Sistemas

¿Qué veremos hoy?

Este módulo cubre los fundamentos para construir, conectar y desplegar aplicaciones web completas. Sigue el hilo de la historia: desde el formulario que ve el usuario hasta el servidor que procesa la información.



3.6 — Aplicación Web Básica

Formularios, manejo de datos y navegación entre páginas.



3.7 — Integración de Componentes

Comunicación entre capas y flujo de información.



3.8 — Despliegue en Entornos

Comparativa entre desarrollo, pruebas y producción.



Objetivo del módulo: que puedas construir y desplegar una aplicación web funcional de extremo a extremo.



3.6 — Aplicación Web Básica

Una aplicación web básica es mucho más que una página estática: es un sistema que recibe entradas del usuario, las procesa y devuelve una respuesta. Para construirla necesitamos tres componentes esenciales que trabajan juntos.



Interfaz (Front-end)

HTML, CSS y JavaScript para lo que el usuario ve e interactúa directamente.



Lógica (Back-end)

Servidor que recibe, valida y procesa los datos enviados por el usuario.



Persistencia (BD)

Base de datos que almacena y recupera la información de forma estructurada.

3.6.1 — Formularios Web

Los formularios son la principal puerta de entrada de datos en cualquier aplicación web. Con el atributo `action` definimos el destino de los datos, y con `method` elegimos el protocolo HTTP. Veamos un ejemplo mínimo pero completo:

```
<!-- Formulario de registro de usuario -->
<form action="/registro" method="POST">
  <label for="nombre">Nombre completo:</label>
  <input type="text" id="nombre" name="nombre" required />

  <label for="email">Correo electrónico:</label>
  <input type="email" id="email" name="email" required />

  <label for="password">Contraseña:</label>
  <input type="password" id="password" name="password" />

  <button type="submit">Registrarse</button>
</form>
```



¿Sabías que...?

El atributo `required` activa la validación nativa del navegador sin necesitar JavaScript. ¡Ahorra código y mejora la accesibilidad!

GET vs POST

- **GET:** datos visibles en la URL. Ideal para búsquedas.
- **POST:** datos en el cuerpo. Ideal para datos sensibles como contraseñas.

3.6.2 — Manejo de Datos

Una vez que el formulario envía los datos, el servidor debe recibirlos, validarlos y procesarlos de forma segura. En este ejemplo usamos JavaScript con Node.js/Express, un stack muy popular en la industria.

```
// Servidor Express - manejo del registro
const express = require('express');
const app = express();
app.use(express.urlencoded({ extended: true }));

app.post('/registro', (req, res) => {
  const { nombre, email, password } = req.body;

  // Validación básica del lado del servidor
  if (!nombre || !email || !password) {
    return res.status(400).send('Todos los campos son obligatorios.');
```

```
  }

  // Aquí iría la lógica de guardado en base de datos
  console.log(`Nuevo usuario: ${nombre} (${email})`);
  res.send('¡Registro exitoso!');
});
```

```
app.listen(3000, () => console.log('Servidor en puerto 3000'));
```

  **Tip de seguridad:** Nunca guardes contraseñas en texto plano. Usa librerías como `bcrypt` para aplicar hashing antes de almacenarlas en la base de datos.

3.6.3 – Navegación Entre Páginas

Navegación Tradicional

Cada enlace realiza una nueva solicitud HTTP al servidor, que devuelve una página HTML completa. Sencillo pero más lento.

```
<nav>
  <a href="/inicio">Inicio</a>
  <a href="/productos">Productos</a>
  <a href="/contacto">Contacto</a>
</nav>
```

Navegación SPA (Single Page App)

El contenido se actualiza dinámicamente con JavaScript sin recargar la página completa. Más fluida y moderna.

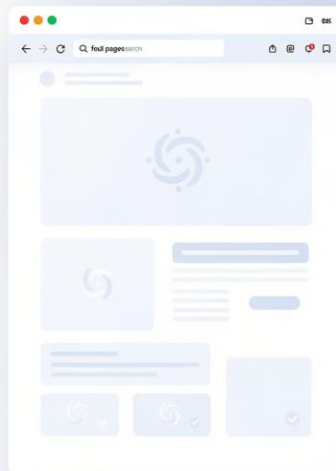
```
// Ejemplo de cómo una SPA recargaría
function navegarTradicional() {
  history.pushState({}, '', '/inicio');
  cargarContenidoTradicional();
}

// Ejemplo de cómo una SPA recargaría
function navegarSPA() {
  history.pushState({}, '', '/inicio');
  // Aquí se actualizaría el contenido de la SPA
  cargarContenidoSPA();
}
```

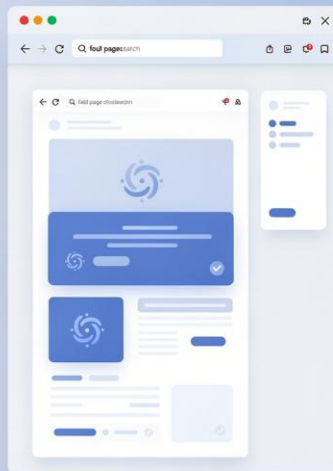
Navegación Tradicional

Recarga completa de página con cada clic

Full Page Reload



Single Page Application



Navegación SPA

Actualiza contenido dinámicamente sin recargar

Momento Interactivo #1

Antes de avanzar a la integración de componentes, revisemos lo visto hasta ahora. Responde esta pregunta en el chat.

¿Cuándo usarías GET y cuándo POST?

Opción A

GET para todos los formularios, porque es más simple de implementar en el servidor.

Opción B

POST cuando los datos son sensibles o cuando modifican el estado del servidor.

Opción C

Depende únicamente del tamaño de los datos que se envían al servidor.

  **Instrucción:** Escribe tu respuesta en el chat con una sola letra (A, B o C) y una justificación breve. ¡Revisamos juntos en 2 minutos!

3.7 – Integración de Componentes

Una aplicación web moderna no es un bloque monolítico: está compuesta por capas especializadas que se comunican entre sí. La integración de componentes es el arte de hacer que estas capas trabajen en armonía, cada una cumpliendo su función sin solaparse con las demás.

→ Separación de responsabilidades

Cada capa tiene un rol claro: presentación, lógica de negocio y acceso a datos.

→ Contratos entre capas (APIs)

Las capas se comunican mediante interfaces bien definidas, lo que facilita el mantenimiento y los cambios.

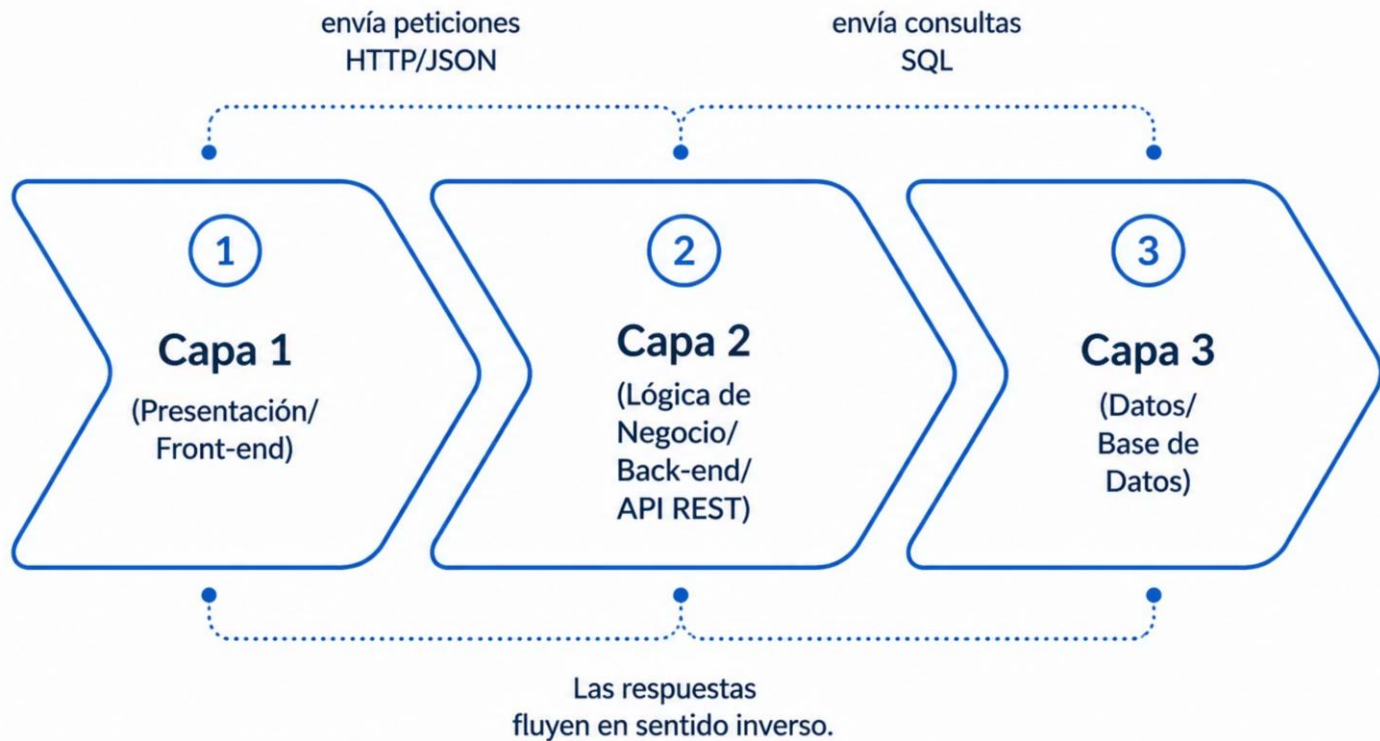
→ Independencia tecnológica

Cada componente puede actualizarse o reemplazarse sin afectar a los demás, siempre que respete el contrato.



3.7.1 – Comunicación Entre Capas

La arquitectura de tres capas (presentación, lógica y datos) es el estándar de la industria. La comunicación entre ellas sigue rutas bien definidas: el front-end nunca accede directamente a la base de datos; todo pasa por el back-end.



```
// Front-end: petición fetch a la API del back-end
fetch('/api/productos')
  .then(res => res.json())
```

3.7.2 – Flujo de Información

Entender el flujo completo de la información — desde que el usuario hace clic hasta que recibe una respuesta — es fundamental para depurar errores y diseñar sistemas robustos. Cada paso tiene su propia responsabilidad.



1. Evento del usuario

Clic, envío de formulario o navegación dispara la petición.



3. Procesamiento

El servidor valida, ejecuta la lógica y consulta la base de datos.



2. Petición HTTP

El navegador envía método, ruta, cabeceras y cuerpo al servidor.



4. Respuesta

El front-end recibe JSON o HTML y actualiza la interfaz.

Momento Interactivo #2

3.8 – Despliegue en Entornos

El despliegue (*deployment*) es el proceso de mover tu aplicación desde tu computadora local hasta un servidor donde otros puedan usarla. Profesionalmente, toda aplicación pasa por al menos tres entornos antes de llegar al usuario final.



Desarrollo (Local)

El desarrollador construye y prueba en su máquina. Máxima libertad, sin consecuencias por errores.



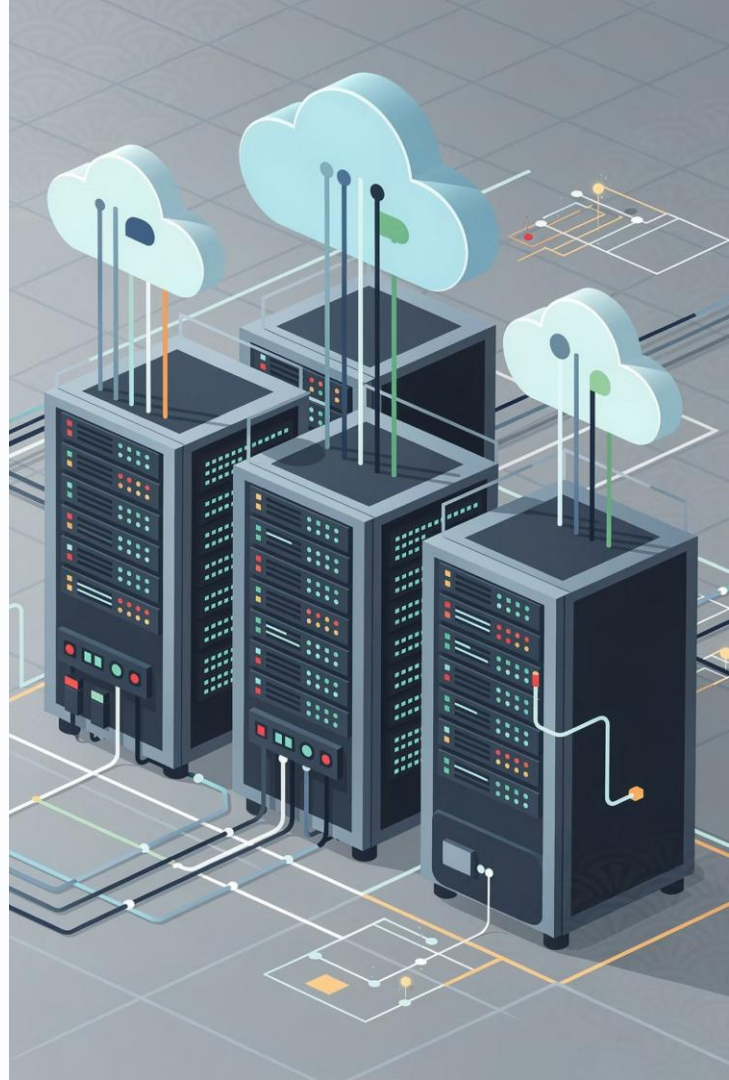
Pruebas (Staging)

Ambiente que replica producción. Aquí se valida que todo funcione antes del lanzamiento.



Producción (Live)

El entorno real donde los usuarios finales acceden a la aplicación. Cero tolerancia a errores.



Comparativa de Entornos de Despliegue

Cada entorno tiene características, propósitos y restricciones distintas. Conocer estas diferencias evita errores comunes como desplegar código sin probar directamente en producción.

Característica	Desarrollo	Pruebas (Staging)	Producción
Objetivo	Construir y codificar	Validar y verificar	Servir al usuario final
Datos	Datos ficticios / locales	Copia anonimizada de prod.	Datos reales de usuarios
Errores	Tolerados y esperados	Deben documentarse	Inaceptables — alertas activas
Acceso	Solo el desarrollador	Equipo interno / QA	Público general
Variables (.env)	NODE_ENV=development	NODE_ENV=staging	NODE_ENV=production
Frecuencia de deploy	Constante (cada cambio)	Por sprint o versión	Planificado y controlado

⚠️ ⚠️ Nunca uses credenciales de producción en tu entorno de desarrollo. Mantén siempre archivos .env separados por entorno y agrégalos al .gitignore.

Variables de Entorno y Configuración

Una de las mejores prácticas del despliegue profesional es separar la configuración del código. Las variables de entorno permiten que el mismo código funcione correctamente en cualquier entorno sin modificaciones.

Archivo `.env` (Desarrollo)

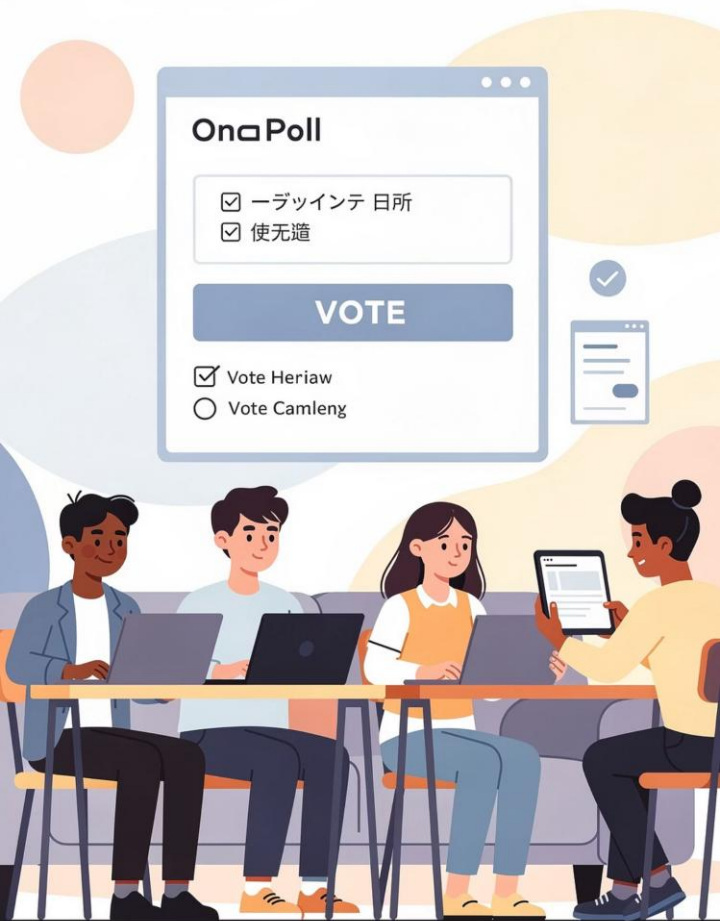
```
# .env.development
NODE_ENV=development
PORT=3000
DB_HOST=localhost
DB_NAME=miapp_dev
DB_USER=root
DB_PASS=1234
API_KEY=test_key_abc123
```

Uso en el código

```
// Carga automática con dotenv
require('dotenv').config();

const config = {
  port: process.env.PORT || 3000,
  db: {
    host: process.env.DB_HOST,
    name: process.env.DB_NAME,
    user: process.env.DB_USER,
    pass: process.env.DB_PASS
  }
};

// El código NO cambia entre entornos
// ¡Solo cambia el archivo .env!
```



⏸ Momento Interactivo #3

Encuesta Rápida

Usa el chat o la herramienta de encuestas de tu plataforma para responder:

1

¿En qué entorno debería detectarse un bug crítico primero?

A) Desarrollo B) Staging C) Producción D) No importa el entorno

2

¿Qué archivo NUNCA deberías subir a GitHub?

A) `package.json` B) `.env` C) `index.html` D) `README.md`

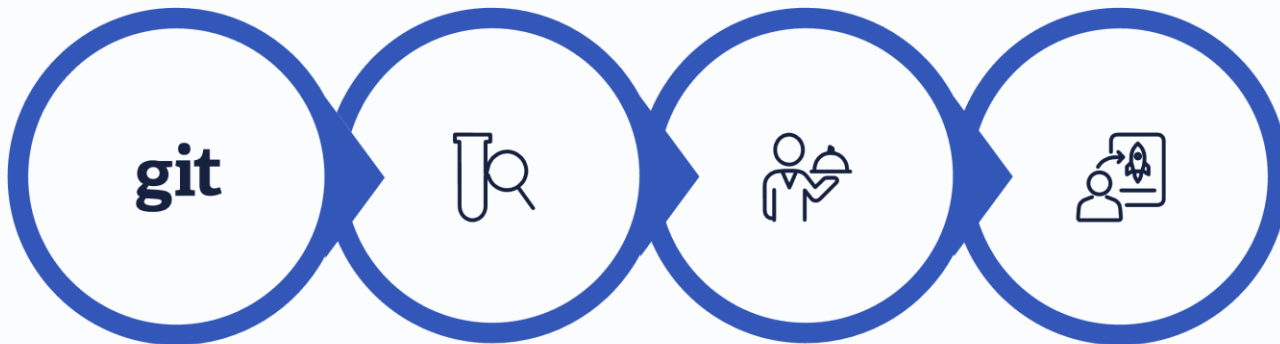
3

¿Cuál es el propósito del entorno de Staging?

A) Reemplazar producción B) Validar antes del lanzamiento C) Solo para diseñadores D) Guardar backups

Flujo Completo de Despliegue CI/CD

Los equipos profesionales automatizan el despliegue mediante pipelines de **Integración Continua / Despliegue Continuo (CI/CD)**. Este flujo garantiza que cada cambio de código pase por todas las validaciones antes de llegar a producción.



Commit &
Push

Pruebas CI

Despliegue
Staging

Aprobación &
Prod



Git Push

El desarrollador hace commit y push al repositorio remoto.



Tests automáticos

El pipeline ejecuta pruebas unitarias y de integración.



Deploy a Staging

Si los tests pasan, se despliega automáticamente en staging.



Producción

Tras aprobación, el pipeline despliega a producción.

Herramientas de Despliegue Populares

El ecosistema de despliegue es amplio. Aquí están las plataformas que más encontrarás en el mercado laboral, desde opciones gratuitas para proyectos estudiantiles hasta soluciones empresariales.



Vercel / Netlify

Ideales para front-end y apps de Node.js. Deploy con un solo `git push`. Plan gratuito generoso — perfectos para proyectos universitarios.



Heroku / Railway

Plataformas PaaS para aplicaciones full-stack. Abstraen la infraestructura y permiten enfocarse en el código.



AWS / GCP / Azure

Las grandes nubes empresariales. Máxima flexibilidad y escala, pero con curva de aprendizaje más pronunciada.

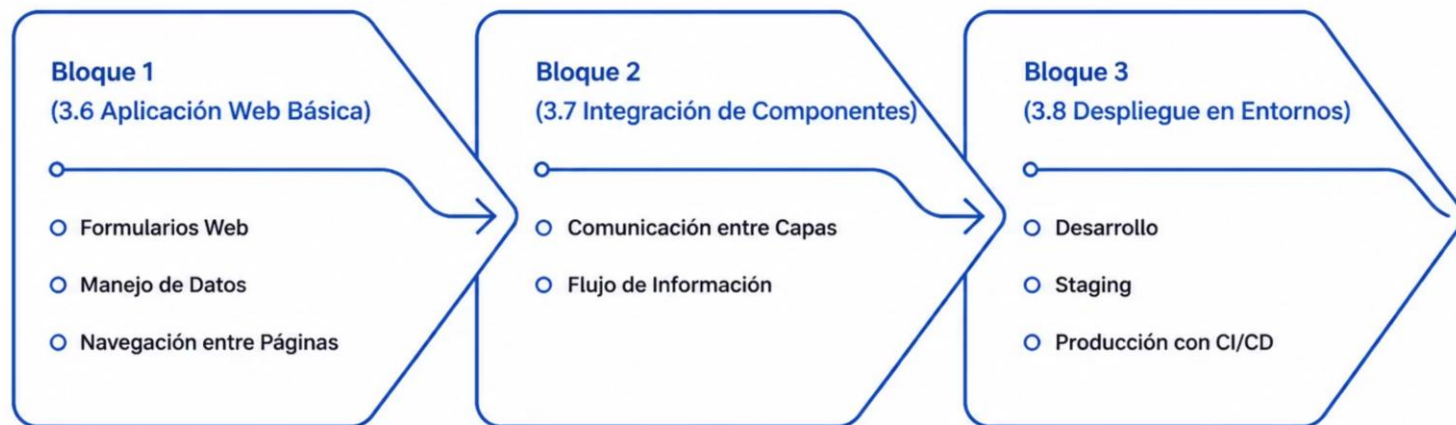


Docker + Contenedores

Empaqueta tu app con todas sus dependencias para garantizar que funcione igual en cualquier entorno.

Resumen Visual del Módulo 3

Cada bloque del módulo encaja con el siguiente para formar el ciclo completo de una aplicación web profesional. De la interfaz al despliegue, ahora tienes el mapa completo.





Buenas Prácticas — Para Llevarte a Casa

Estas son las reglas de oro que diferencian el código de un estudiante del código de un profesional. Aplícalas desde hoy en todos tus proyectos.

✓ Valida siempre en el servidor

La validación del cliente (HTML5/JS) mejora la experiencia, pero nunca es suficiente. Un usuario malicioso puede saltársela fácilmente.

✓ Separa configuración del código

Usa variables de entorno (`.env`) para credenciales, URLs y claves. Nunca hardcodees valores sensibles en el código fuente.

✓ Prueba antes de desplegar

Staging existe por una razón. Un bug en producción puede costar usuarios, dinero y reputación — errores que se evitan con una prueba de 10 minutos.

✓ Documenta tu API

Los contratos entre capas deben estar documentados. Una API bien documentada reduce el tiempo de integración y facilita el trabajo en equipo.